Thadomal Shahani Engineering College, Bandra Department of Artificial Intelligence & Data Science

Year: 2026-2027 Sem V-NEP
Course: Agile Software Development & DevOps Lab
Class: T12
Date of Submission: 06/10/2026

A Mini Project Report on

IntentWay

A Context-Aware, Dynamic Multi-Stop Intent Routing Engine Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course
Agile Software Development & DevOps Lab
Semester V (NEP) · Academic Year 2026–2027 · Class T12

Submitted by

Name	Roll No.
Adnan Khan	2401047
Ashhad Khan	2401048
Prakriti Ghanekar	2401030

Under the guidance of
Prof. Sanober Shaikh
Department of AI&DS
Thadomal Shahani College of Engineering
2026–2027

CERTIFICATE

This is to certify that the mini project entitled “IntentWay: A Context-Aware, Dynamic Multi-Stop Intent Routing Engine” is a bona fide work of Adnan Khan (2401047), Ashhad Khan (2401048) and Prakriti Ghanekar (2401030), submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

The work demonstrates the application of Agile planning, Git/GitHub based version control, Docker and Docker Compose based environment management, Jenkins based continuous integration, and Nagios based availability monitoring to a single software product.

Project Guide	**Head of Department**
Sanober Shaikh	Madhuri Rao
Date:06/10/2026	

ABSTRACT

IntentWay is a context-aware, dynamic multi-stop intent routing engine designed to convert a natural-language travel requirement into an executable road route containing relevant intermediate locations. Instead of asking the user to manually search for stops, the system interprets the stated intent, identifies the origin and destination, generates a base road route, discovers relevant points of interest (POIs) around that route, filters candidates that fall within the usable corridor, and then selects and orders stops using a greedy multi-stop routing strategy before requesting a final road route.

The project was developed as a Semester V Agile Software Development & DevOps Lab mini project. The software work was tracked in Jira using nine completed work items, with the second sprint covering the solver, routing wizard, map presentation, Docker Compose, Jenkins CI, and Nagios monitoring tasks. The implementation is hosted in GitHub using feature branches and merge-based integration. The current repository contains separate branches for the intent parser, spatial filtering, route solver, presentation work, Nagios monitoring, and a fix branch used to validate the automated test pipeline.

The application combines a React/Vite frontend with a FastAPI backend. Routing and discovery rely on Nominatim for geocoding, OSRM for road routes, Overpass for dynamic OpenStreetMap POI discovery, Shapely and PostGIS for spatial corridor filtering, and React-Leaflet for map presentation. Docker Compose provides the repeatable local runtime consisting of the frontend, backend, PostGIS database, and Nagios monitoring service. Jenkins executes the repository pipeline using a Python 3.11 based Linux container. In the demonstrated successful build, Jenkins checked out the repository, created a Python virtual environment, installed backend dependencies, executed 94 backend tests, and built the frontend with Vite; the run completed successfully with one non-failing deprecation warning.

The captured DevOps evidence shows the completed Jira work items, GitHub branch and collaborator configuration, Git history, Docker service status, an SCM-triggered Jenkins build, and Nagios Tactical Overview. At the time of capture, Nagios reported four monitored hosts as UP and nine services as OK, with one service in WARNING state. These artifacts demonstrate the intended loop of planning → version control → automated validation → containerized execution → monitoring.

Keywords: Agile, Scrum, Git, GitHub, Docker, Docker Compose, Jenkins, CI, FastAPI, React, PostGIS, Shapely, Nominatim, OSRM, Overpass, Nagios, Computational Geometry, Route Optimization

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

A common weakness in route-planning interfaces is that the user has to translate a real-world requirement into a sequence of manual map searches. A request such as “find a pharmacy on my way from Wadala to Bandra” contains at least four separate computational tasks: understanding the intent, identifying the geographic endpoints, discovering candidate places, and choosing a stop that adds as little detour as possible. A useful system therefore needs to connect natural-language interpretation with geographic computation rather than treating them as separate screens.

IntentWay was developed to address this interaction problem. The project combines deterministic intent parsing with spatial computation and external routing services. The backend separates responsibilities into geocoding, route generation, POI discovery, spatial filtering, and route optimization modules. The frontend presents the final route and candidate stops on an interactive map and provides a journey simulation so that the route can be inspected as a sequence rather than as a static line.

The Agile Software Development & DevOps Lab provided the delivery framework. The team treated the application as a single product while applying the lab practices to its development: requirements were tracked in Jira, source code was versioned in Git and GitHub, work was performed on dedicated branches, Docker and Docker Compose provided repeatable environments, Jenkins validated repository changes, and Nagios monitored service availability. This made the tool chain part of the project itself rather than a separate demonstration.

1.2 Problem Statement

To design and implement a web application that accepts a natural-language route intent, converts that intent into a set of spatial constraints, discovers relevant candidate POIs along a route corridor, chooses an efficient multi-stop ordering, and displays the resulting road route, while delivering the software through an Agile and DevOps workflow that provides version control, automated validation, containerization, and monitoring.

1.3 Objectives

1. Build a deterministic intent parser that maps natural-language requirements to canonical categories such as pharmacy, restaurant, hospital, and other supported POI types.
2. Convert textual origins and destinations into geographic coordinates and generate a base road route before POI discovery.
3. Discover relevant POIs dynamically around the route and filter candidates to those that are spatially close to the route corridor.
4. Implement a greedy multi-stop solver that selects and orders candidate stops using detour-aware route information.
5. Display candidate locations, selected stops, the final route, route metadata, and journey simulation in an interactive web interface.

6. Use Jira to plan and track the work and Git/GitHub to maintain a traceable branch-and-merge history.
7. Containerize the application with Docker Compose so that the frontend, backend, database, and monitoring environment can be started consistently.
8. Use Jenkins to automatically validate repository changes and Nagios to monitor service availability in the running Docker environment.

1.4 Scope

The current scope is focused on route planning and contextual stop selection. The system handles point-to-point route requests with one or more intended intermediate stops and demonstrates the complete computational chain from natural language to map visualization. It is designed as a local or single-host deployment and uses publicly accessible OpenStreetMap based services. Online user authentication, payment processing, long-term traffic prediction, and production-scale distributed deployment are outside the current scope.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in IntentWay	Section
1	Version control with Git and GitHub	Feature branches, merge history, GitHub repository, collaborators, and traceable commits	5.1
2	Containerisation with Docker	Separate frontend/backend images and a custom Jenkins image for CI tooling	5.2
3	Docker Compose	Frontend, FastAPI backend, PostGIS and Nagios connected on a dedicated network	5.3
4	Jenkins CI pipeline	SCM-triggered checkout, Python environment, backend tests and frontend build	5.4
5	Deployment validation	Repeatable Docker Compose start-up and verification of running service health	5.5
6	Agile project lifecycle	Jira backlog, sprint planning, assignments and Done states	2
7	Monitoring	Nagios host/service availability checks and Tactical Overview	5.6

Table 1.1: Mapping of lab experiments to IntentWay

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main interaction
Traveller / User	Express a route-related intent without manually constructing every stop	Enter origin, destination and intent; inspect candidates; view final route and simulation
Planner / Developer	Translate user intent into deterministic and testable routing operations	Maintain parser, filtering, solver, routing and orchestration modules

Role	Goal	Main interaction
DevOps / Maintainer	Keep the application reproducible and observable	Run Docker Compose, validate Jenkins builds, and inspect Nagios monitoring

Table 2.1: User roles and responsibilities

2.2 Functional Requirements

- The user shall be able to enter an origin and a destination as textual locations.
- The user shall be able to express an intermediate requirement in natural language, for example requesting a pharmacy or restaurant on the route.
- The backend shall parse the intent into a canonical category and reject unsupported or ambiguous requests in a controlled manner.
- The system shall geocode origin and destination and generate a road route before searching for intermediate candidates.
- The system shall discover relevant POIs along the route corridor and provide candidate names and coordinates.
- The system shall filter candidates geographically and allow the solver to operate only on candidates available in the data returned by the discovery stage.
- The solver shall select and order intermediate stops to reduce unnecessary detour relative to the available candidates.
- The frontend shall display the base route, candidate POIs, selected stops, and final multi-stop route on an interactive map.
- The application shall expose a journey simulation based on route coordinates so that the route can be inspected progressively.
- The DevOps setup shall provide automated backend testing, frontend build validation, repeatable Docker execution, and service monitoring.

2.3 Non-Functional Requirements

- Correctness: deterministic parsing, explicit validation, stable response models, and test coverage for solver and spatial filtering logic.
- Reliability: backend and database services should be health-checked before dependent services are considered ready.
- Portability: the core application should be runnable with Docker Compose on another machine with Docker support.
- Maintainability: routing concerns are separated into modules rather than combined into one endpoint implementation.
- Observability: the infrastructure layer should expose service availability through Nagios.
- Reproducibility: Jenkins should use the same Python and Node major-version assumptions as the tested project environment.
- Safety of changes: changes should be made on branches and integrated into main only after validation.

2.4 Scrum Setup

The Jira project named IntentWay was used to make the work visible at issue level. The captured List view contains nine completed work items, IW-1 through IW-9. The items progress from project foundation and deterministic parsing through spatial filtering, multi-stop solving, map presentation, Docker Compose, Jenkins CI, and Nagios monitoring. The second sprint screenshot shows five completed items—IW-4 through IW-9—with no remaining work visible in the backlog at the time of capture.

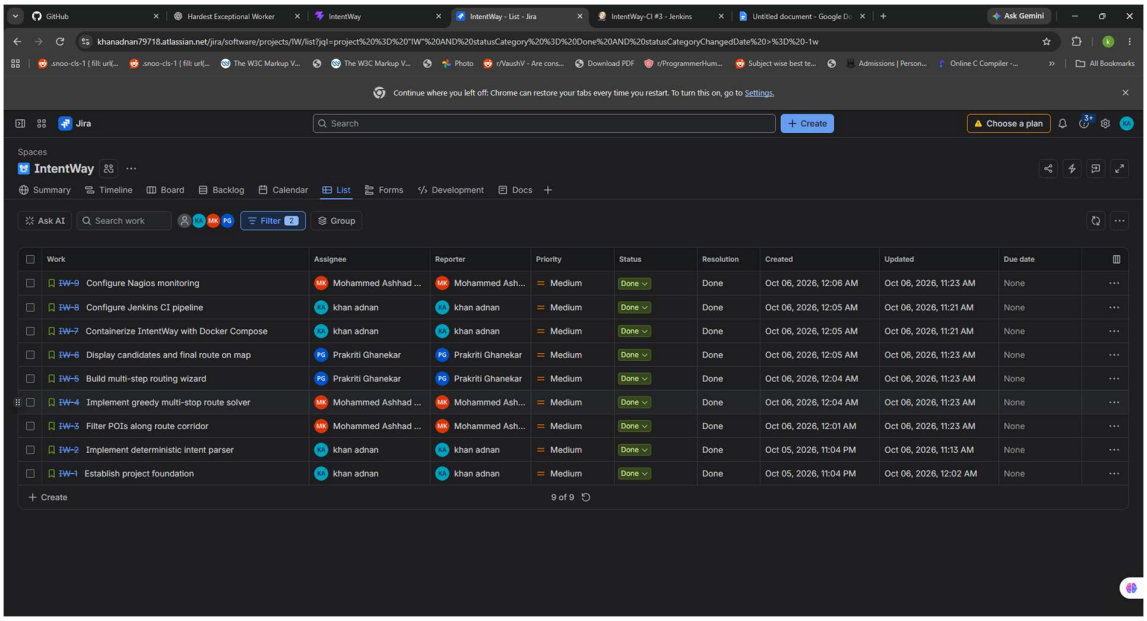


Figure 2.1: IntentWay Jira list showing nine completed work items and assignees

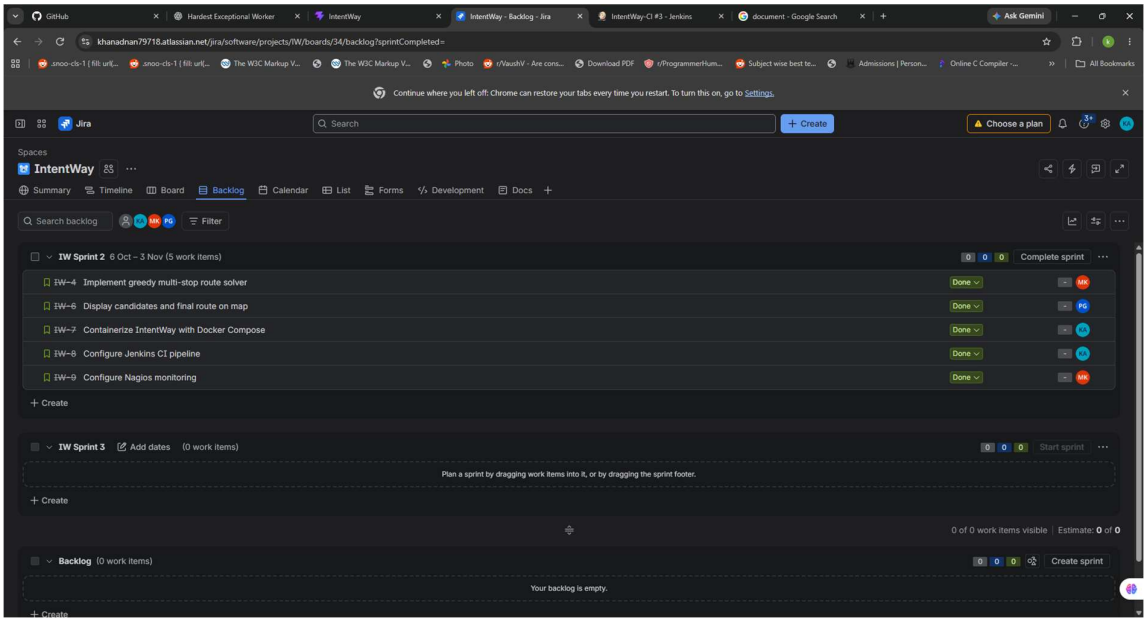


Figure 2.2: Jira backlog view showing IW Sprint 2 and its completed work items

2.5 Product Backlog

Key	Work item	Assignee	Status
IW-1	Establish project foundation	Adnan Khan	Done
IW-2	Implement deterministic intent parser	Adnan Khan	Done
IW-3	Filter POIs along route corridor	Ashhad Khan	Done
IW-4	Implement greedy multi-stop route solver	Ashhad Khan	Done
IW-5	Build multi-stop routing wizard	Prakriti Ghanekar	Done
IW-6	Display candidates and final route on map	Prakriti Ghanekar	Done
IW-7	Containerize IntentWay with Docker Compose	Adnan Khan	Done
IW-8	Configure Jenkins CI pipeline	Adnan Khan	Done
IW-9	Configure Nagios monitoring	Ashhad Khan	Done

Table 2.2: IntentWay product backlog captured from Jira

2.6 Sprint Plan

Sprint	Primary scope	Completed items
Sprint 1	Foundation and core geographic intent flow	IW-1, IW-2, IW-3
Sprint 2	Optimization, map integration, containerization and DevOps	IW-4, IW-5, IW-6, IW-7, IW-8, IW-9

Table 2.3: Sprint scope derived from the Jira work sequence

2.7 Definition of Done

1. The code for the work item is implemented in the repository and the corresponding feature/fix branch is identifiable.
2. The change is committed and pushed to GitHub so that the source and its history are reproducible.
3. The Jenkins pipeline can check out the branch and execute the configured validation stages successfully.
4. Automated backend tests pass and the frontend production build completes successfully.
5. The feature or infrastructure change is verified in the running Docker environment where applicable.
6. For monitoring work, the expected host/service state is visible in Nagios.

3. System Design

3.1 Architecture

IntentWay is organized as a modular routing pipeline. The browser-based React application sends a route planning request to the FastAPI backend. The backend first resolves the textual origin and destination, requests a road route, and then uses the route geometry as the spatial context for discovering and filtering candidate POIs. The solver operates after candidate discovery and produces an ordered set of stops. A final road route is then requested for the

origin, selected stops, and destination. This separation makes it possible to test the intent parser, spatial filter, solver, and routing layer independently.

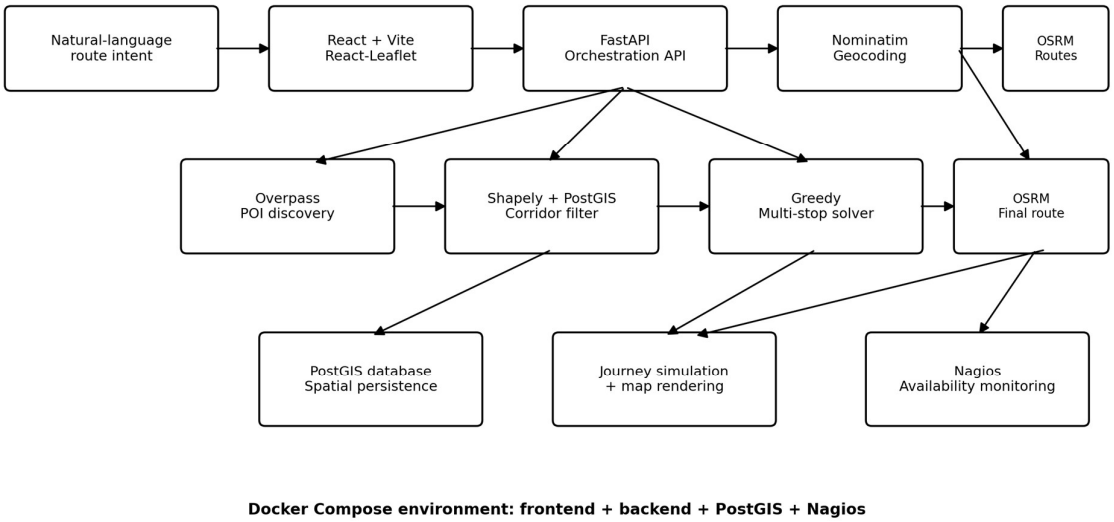


Figure 3.1: IntentWay system and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	React, Vite, React-Leaflet, Leaflet	Component based UI, fast development build, and interactive route visualization
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic Settings	Typed request models, modular API design, and fast local execution
Spatial computation	Shapely, PostGIS	Geometric corridor tests and spatial querying around a route
Geocoding	Nominatim	Convert human-readable locations to coordinates
Routing	OSRM	Road-aware route generation for base and final multi-stop paths
POI discovery	Overpass / OpenStreetMap	Dynamic discovery of nearby pharmacies, restaurants and other supported POIs
DevOps	Git, GitHub, Docker, Docker Compose, Jenkins, Nagios	Versioning, reproducibility, automated validation and availability monitoring

Table 3.1: Technology stack used in IntentWay

3.3 Spatial Data and Route Model

The core spatial object is the route geometry. A base route is represented as a sequence of coordinates returned by a road routing service. The spatial filter converts the route into a corridor and checks whether candidate POIs intersect or fall within the corridor. PostGIS provides a persistent spatial database layer while Shapely is used for geometric operations in Python. This allows the system to reason about distance from the route rather than relying only on textual location matching.

3.4 Intent-to-Route Lifecycle

1. Accept the user request containing origin, destination, and an intermediate intent.
2. Parse the text deterministically into a supported POI category.
3. Geocode the origin and destination.
4. Generate the base road route between the endpoints.
5. Discover dynamic POIs along or around the route corridor.
6. Apply spatial filtering to discard candidates that are not sufficiently close to the corridor.
7. Run the greedy multi-stop solver over the remaining candidate set.
8. Request the final road route with the selected stop coordinates as waypoints.
9. Return structured route data, candidate information, and simulation data to the frontend.
10. Render the route and progressively simulate movement along its coordinates.

3.5 API Design

Method	Endpoint / Operation	Purpose
GET	/health	Report service health and confirm the API process is responding
POST	/api/route/plan	Orchestrate geocoding, base routing, POI discovery, filtering, solving, final routing and response creation
Internal	Intent parser module	Map natural-language terms to canonical supported intent categories
Internal	Spatial filter module	Build route corridor and query/filter candidate POIs
Internal	Solver module	Select and order candidate stops to reduce unnecessary detour
Internal	Routing / POI modules	Call OSRM and Overpass integrations with controlled response parsing

Table 3.2: Core IntentWay API and service operations

3.6 Reliability and Security Considerations

- Input requests are validated using typed Pydantic models before entering the routing pipeline.
- External API calls are isolated behind dedicated modules so failures can be surfaced as controlled application errors instead of leaking transport details to the frontend.
- Database connection parameters are supplied through environment variables in the Docker Compose environment.
- The backend container is isolated from the host through Docker networking and only the required port is published.
- The monitoring layer is kept separate from the business logic so monitoring configuration can change without modifying route computation.

4. Application Implementation

The application implementation is deliberately split by responsibility. The project started from a minimal repository foundation and then expanded through deterministic parsing, spatial

filtering, route solving, real map routing, journey simulation, containerization, Jenkins validation and monitoring. This modular organization is also what makes it possible to test the solver and spatial components independently from live external routing services.

4.1 Deterministic Intent Parser

The intent parser is intentionally deterministic rather than model-dependent. It detects supported keywords and phrases and maps them to canonical categories. This choice makes the project easy to test, reproducible across runs, and straightforward to debug during a classroom demonstration. The parser also provides a stable interface to the rest of the pipeline: the spatial and routing modules operate on the canonical category instead of reinterpreting raw text.

4.2 Spatial Filtering

The spatial filter receives route coordinates and candidate POIs and evaluates whether each candidate is close enough to the travel corridor to remain relevant. A straight geometric corridor is formed from the route geometry and the filter uses spatial intersection logic to remove candidates that would require an unreasonable deviation. The module supports the categories needed by the intent parser and is designed so the filtering threshold can be changed without rewriting the solver.

4.3 Dynamic POI Discovery

Dynamic discovery expands the candidate set beyond a small hard-coded database. The application queries OpenStreetMap data through Overpass for POIs near the route. The discovery layer converts returned nodes, ways or relations into the internal POI representation used by the spatial filter and solver. This makes the route results depend on currently available map data while keeping the downstream optimization logic deterministic.

4.4 Greedy Multi-Stop Route Solver

The solver operates on the candidate POIs that remain after filtering. It estimates the additional cost of including candidate stops and greedily chooses an ordering that keeps the resulting trip practical. The solver is intentionally designed as a tractable heuristic rather than an exact global optimization algorithm. Its role is to find a strong route ordering from the data currently available to the application; it does not create POIs that were not returned by the discovery stage.

4.5 Map Presentation and Journey Simulation

The frontend presents origin, destination, candidate POIs, selected stops and the final route using React-Leaflet. The map provides visual confirmation that the selected places lie near the intended route. The journey simulation uses route coordinates to move progressively through the computed path, making the multi-stop sequence easier to demonstrate during the final presentation.

4.6 Containerized Application Runtime

The Docker Compose stack used in the project contains a React frontend, the FastAPI backend, PostgreSQL with PostGIS, and Nagios. The frontend and backend communicate over the project network. PostgreSQL persists data through a named volume and initializes the database from the project SQL file. Nagios mounts the project monitoring configuration so that the monitored hosts and services remain versioned with the repository configuration.

5. DevOps Implementation

This chapter explains how the seven lab experiments were applied to IntentWay. The delivery loop begins with a Jira work item, continues through a Git feature branch and GitHub, is validated by Jenkins, is executed through Docker, and is observed through Nagios. The evidence below is taken from the captured project screens and the successful Jenkins run.

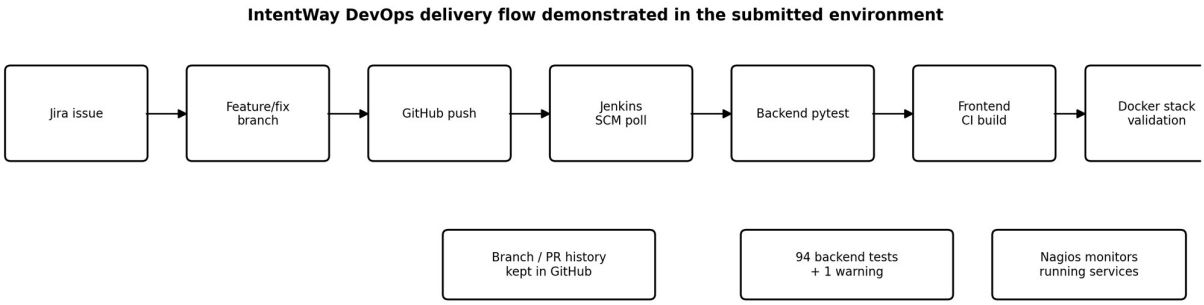


Figure 5.1: IntentWay delivery pipeline from Agile planning to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

IntentWay is hosted in the GitHub repository KhanAdnan20-dev/ASD-D-MINIPROJECT. The project uses separate branches for major feature areas and fixes, including feature/intent-parser, feature/spatial-filter, feature/route-solver, feature/intentway-presentation, feature/nagios-monitoring, develop, main, and fix/api-test-mock. The branch structure supports isolated work while the main branch represents the integrated state of the project.

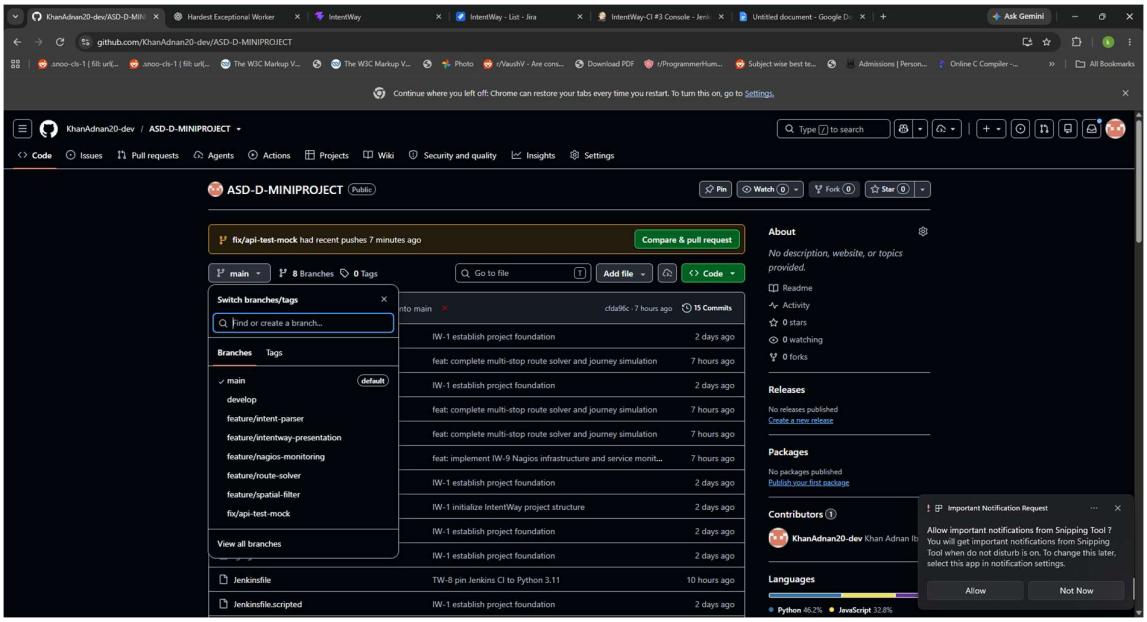


Figure 5.2: GitHub repository showing the main branch and project branches

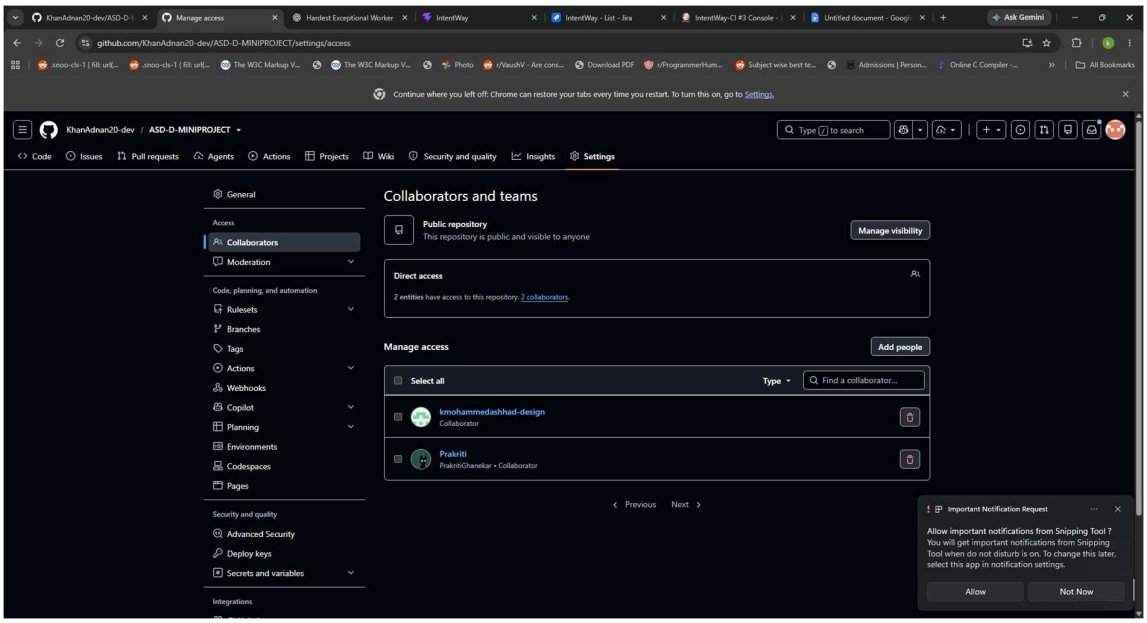


Figure 5.3: GitHub collaborator access for the repository

The Git history is used as the technical record of integration. Important merges include the route solver, Nagios monitoring, presentation integration and the final merge of develop into main. The branch shown in the current evidence also demonstrates how a test fix can be developed separately and pushed without modifying the stable main branch directly.

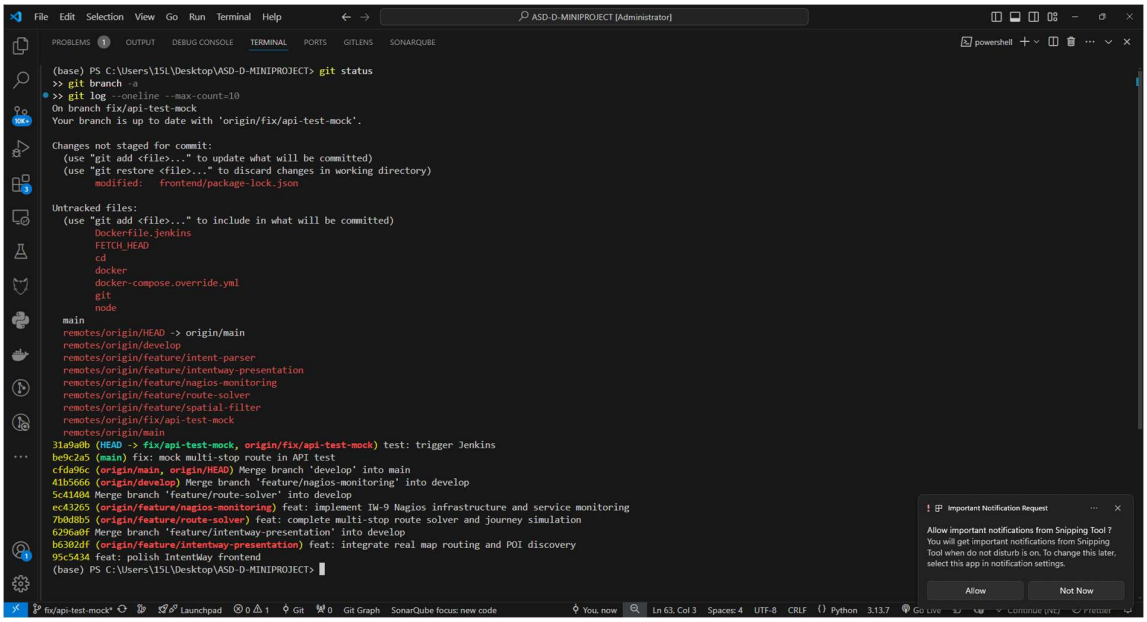


Figure 5.4: Git branch status and recent commit history in the project workspace

Branch / commit	Purpose
main	Integrated stable project branch
develop	Integration branch used before main merge
feature/intent-parser	Deterministic intent parsing
feature/spatial-filter	Spatial corridor filtering
feature/route-solver	Multi-stop route solver and journey simulation
feature/intentway-presentation	Real map routing and POI discovery integration
feature/nagios-monitoring	Nagios monitoring infrastructure and service checks
fix/api-test-mock	Test isolation fix used to validate Jenkins after one endpoint test failed

Table 5.1: Important branches used during IntentWay development

5.2 Containerisation with Docker (Experiment 2)

Docker is used for both the application runtime and the Jenkins execution environment. The backend Dockerfile pins the runtime to Python 3.11 so that the CI environment is consistent with the repository test assumptions. The frontend uses a Node-based build container, and the final development environment includes a separate Jenkins container containing Python 3.11, Node 22, npm and Git. This approach avoids depending on whatever Python version happens to be installed on the host machine.

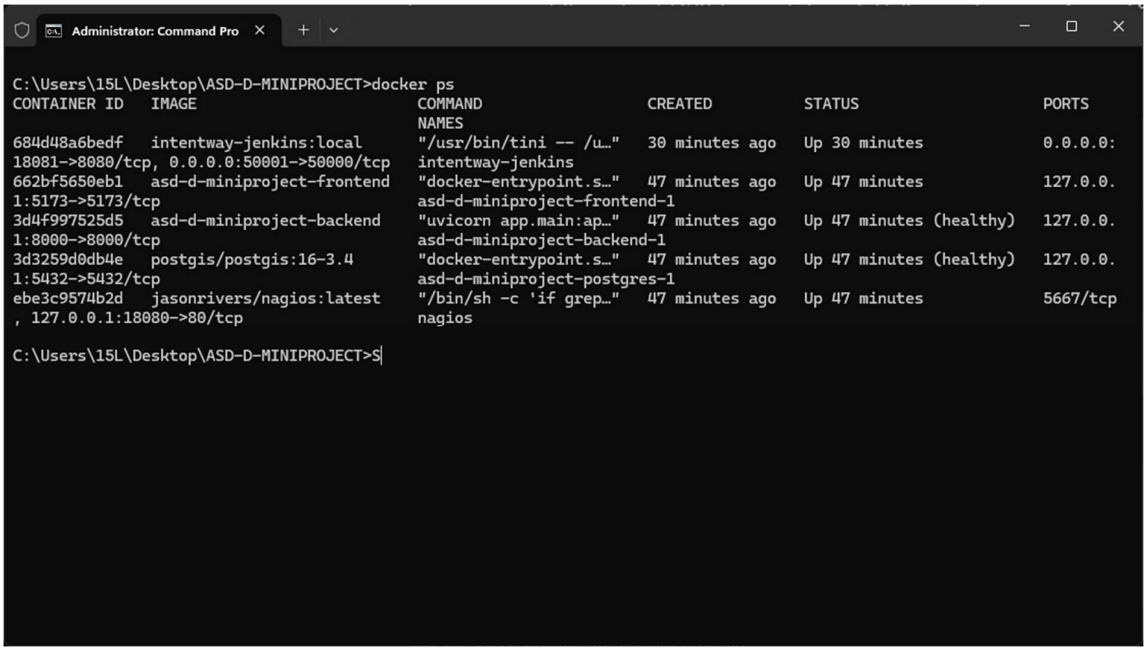


Figure 5.5: Docker containers running the IntentWay stack and the local Jenkins instance

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

Docker Compose defines the IntentWay runtime as a connected multi-service system. The backend waits for PostgreSQL to become healthy, the frontend depends on the backend health state, and all application services join the same intentway-network bridge network. A named Postgres volume preserves the database state across restarts. The screenshot also demonstrates that Nagios exposes its web interface through a host port selected to avoid collisions with existing services on the college computer.

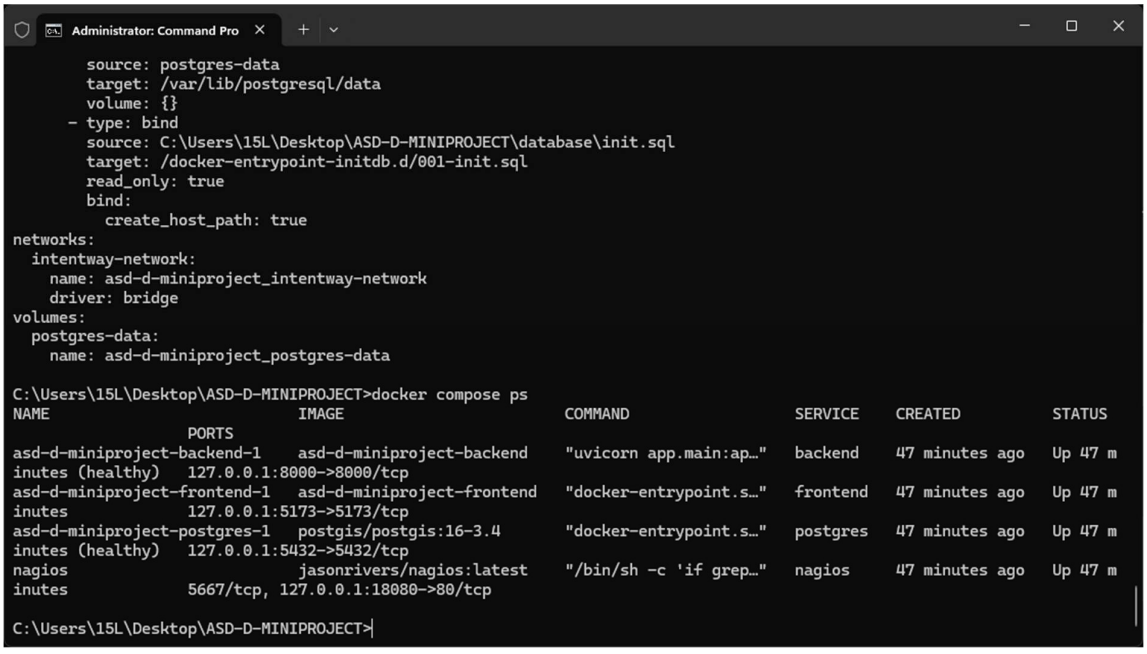


Figure 5.6: Docker Compose configuration and running service status

Service	Container / image role	Host port in demonstrated setup
Frontend	React/Vite application	5173
Backend	FastAPI + Uvicorn application	8000
PostgreSQL	PostGIS 16 / 3.4 database	5432
Nagios	Nagios Core monitoring service	18080
Jenkins (separate)	Custom local CI controller image	18081

Table 5.2: IntentWay runtime services and demonstrated local ports

5.4 Continuous Integration with Jenkins (Experiment 4)

The Jenkins job created for the project is named IntentWay-CI. The pipeline is stored in the repository rather than being written only in the Jenkins UI. Two pipeline files are present: Jenkinsfile for Windows execution and Jenkinsfile.scripted for Linux execution. The college-laptop Jenkins instance runs inside a Linux Docker container, so the scripted pipeline was selected. Its stages are Checkout, Backend dependencies and tests, and Frontend dependencies and build.

Stage	Implementation	Purpose
Checkout	checkout scm	Fetch the exact branch revision from GitHub
Backend dependencies and tests	python3 -m venv; pip install -r backend/requirements.txt; pytest backend/tests	Create an isolated Python environment and validate backend code
Frontend dependencies and build	npm ci --prefix frontend; npm run build --prefix frontend	Install locked frontend dependencies and produce a production build

Table 5.3: Jenkins pipeline stages used by IntentWay

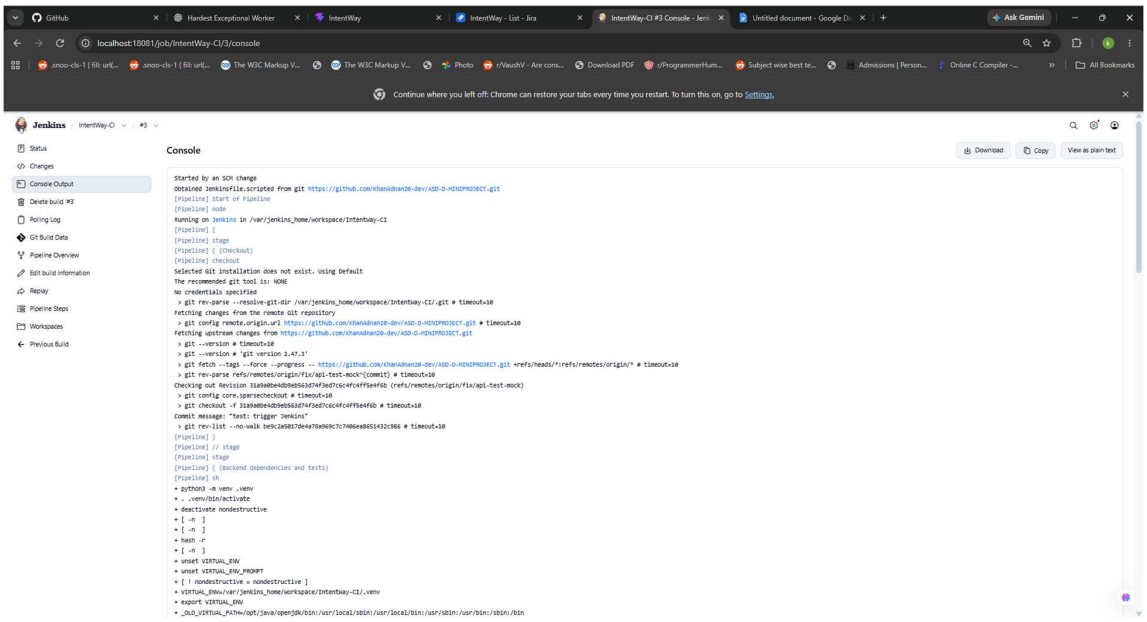


Figure 5.7: Jenkins build started by an SCM change and checking out the fix branch

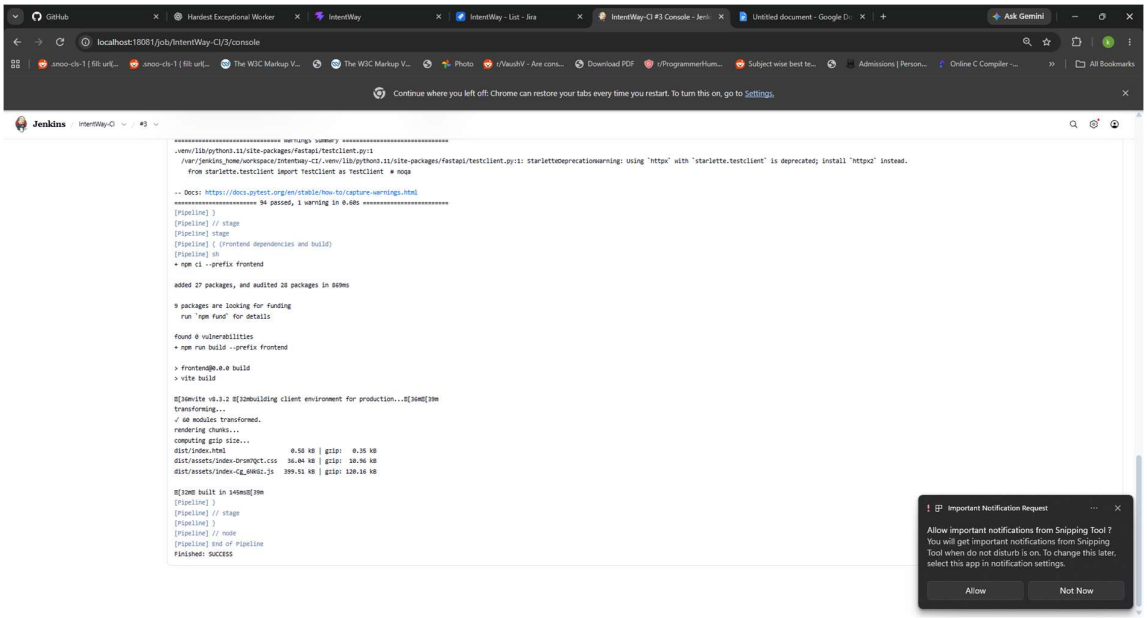


Figure 5.8: Jenkins build completed successfully with 94 tests passed and the frontend build completed

The successful Jenkins run demonstrates a complete branch validation cycle. The log shows the repository being checked out at the pushed fix commit, a Python 3.11 virtual environment being created, all backend dependencies being installed, 94 tests passing with one non-blocking warning, and the Vite frontend build completing successfully. The build ends with “Finished: SUCCESS”.

5.5 Deployment Validation (Experiment 5)

The current project demonstrates repeatable deployment through Docker Compose rather than an unattended production release system. After building and validating the repository, the same project can be brought up using docker compose up --build, and docker compose ps provides a compact health view of the four application services. The separation between CI validation and local deployment keeps the project reproducible without claiming a production CD system that is not currently part of the repository.

The team also used a local Docker override to avoid port conflicts on the college machine. Jenkins occupied port 8080 and an existing Tomcat installation occupied port 8081, so the Nagios host port was moved to 18080. This change was kept local and outside the main project configuration so the repository remained portable.

5.6 Monitoring with Nagios (Experiment 7)

Nagios was introduced as the availability-monitoring layer of IntentWay. Its role is different from application-level logs: it answers whether monitored hosts and services are available at the time of the check. The monitoring configuration is mounted into the Nagios container and the service joins the same Docker network as the application containers, allowing checks to refer to service names rather than relying on host-specific container IP addresses.

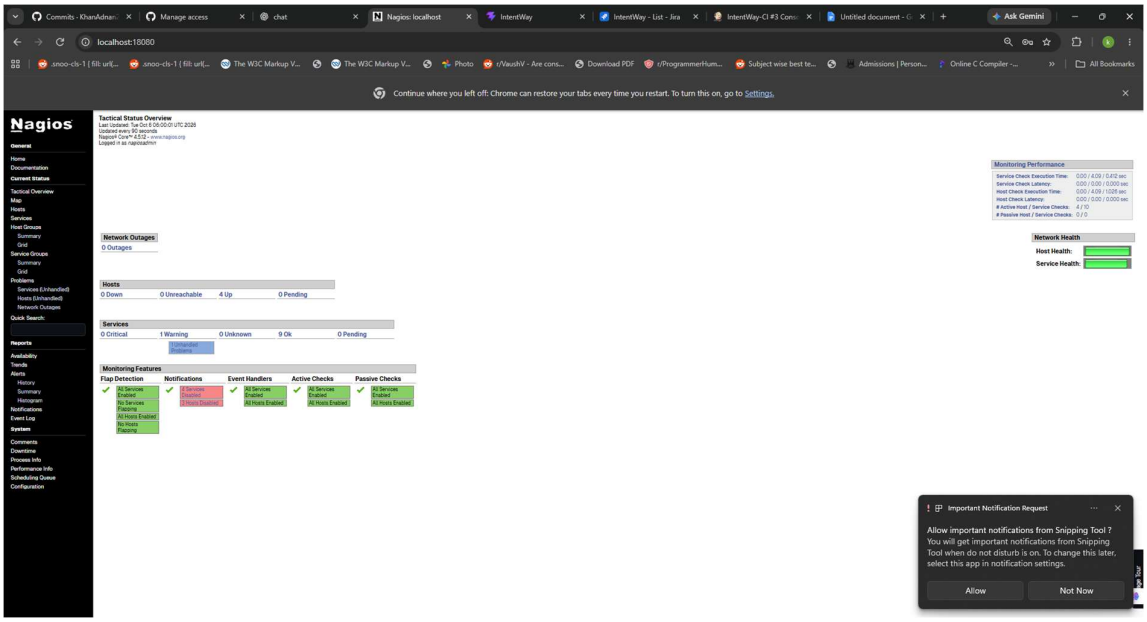


Figure 5.9: Nagios Tactical Overview showing four monitored hosts UP and nine services OK

At the time the screenshot was captured, the Tactical Overview reported 0 hosts down, 0 unreachable, 4 hosts UP, 0 pending, 0 critical services, 1 warning, 0 unknown, and 9 services OK. This is strong evidence that the monitoring configuration is active rather than merely installed. The single warning is visible as a non-OK service state and can be investigated from the Current Status → Services page when required.

Monitoring object	Purpose
Host availability	Detect whether configured application/monitoring hosts are reachable
Backend health	Check whether the API service responds as expected
Frontend availability	Check whether the user-facing web service is reachable
PostgreSQL availability	Detect database connection-level failure
Nagios self-monitoring	Confirm that the monitoring process and web service remain available

Table 5.4: IntentWay monitoring responsibilities

6. Testing

6.1 Automated Tests

The backend contains automated tests for the API, models, intent parser, solver and spatial filtering components. The Jenkins run collected 94 tests. During the first run after configuring the new CI environment, one API route-planning test failed because the test mocked the base routing call but allowed the final multi-stop routing function to make a real external request. The production code was not changed for this problem; the test was corrected by mocking get_multi_stop_route so that the test remained deterministic. The subsequent Jenkins build passed all 94 tests.

Validation	Result	Observation
Backend pytest	94 passed, 1 warning	Warning was non-blocking and related to the Starlette/httpx TestClient deprecation message

Validation	Result	Observation
Frontend lint	0 warnings, 0 errors	Oxlint completed successfully
Frontend build	SUCCESS	Vite transformed 60 modules and produced the production bundle
Docker Compose config	SUCCESS	Compose file and local override resolved correctly
Running stack	Backend/Postgres healthy	Containers reported healthy through Docker health checks
Nagios Tactical Overview	4 UP, 9 OK, 1 WARNING	Availability checks were actively running

Table 6.1: Verification results from the demonstrated IntentWay environment

6.2 Integration Testing

Integration testing for the route-planning flow focuses on the interaction between parsing, geocoding, routing, candidate discovery, spatial filtering, solving, and final route generation. The primary orchestration endpoint is exercised with external geospatial calls mocked in the test suite so that the response is reproducible. The fix branch demonstrates the value of this isolation: a real network call inside a test could otherwise change the result and make a deterministic test fail.

6.3 Deployment and Monitoring Checks

The Docker environment provides another test layer. Once the containers are started, docker compose ps is used to verify that the service graph has reached its expected state. The backend and database are especially important because they expose health states used by Compose dependencies. Nagios then provides an independent monitoring view and shows the service state from the perspective of a monitoring system rather than from the Docker CLI alone.

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Natural-language route intent	Delivered through a deterministic intent parser with canonical supported categories
Route-aware POI discovery	Delivered through base routing, Overpass discovery and corridor filtering
Multi-stop route optimization	Delivered through greedy stop selection and final OSRM routing
Map presentation and simulation	Delivered through React-Leaflet route display and journey simulation
Agile planning	Nine Jira work items captured; the supplied screenshots show all listed items as Done
Version control	Git/GitHub branch-based workflow with integrated main branch and dedicated feature/fix branches
Containerization	Docker Compose runs frontend, backend, PostGIS and Nagios; Jenkins is separated as CI infrastructure
Continuous integration	Jenkins automatically validates the repository; demonstrated successful build passes 94 backend tests and frontend build
Monitoring	Nagios Tactical Overview reports 4 monitored hosts UP and 9 services OK at capture time

Table 7.1: Results against project objectives

7.2 The Feedback Loop in Practice

The most concrete example of the feedback loop occurred during Jenkins validation. The first pipeline execution exposed a failing route-planning endpoint test with an unexpected distance value. Inspection showed that the test mocked the base route but did not mock the final multi-stop route call. Because the latter could execute through the real routing function, the returned distance differed from the expected test fixture. The issue was corrected by isolating that call in the test with a mock. The corrected change was committed on a dedicated branch, pushed to GitHub, and then validated by Jenkins. The subsequent build completed successfully with 94 passed tests and the frontend build stage also passed. This demonstrates how the CI system detects a problem, the problem is repaired in a controlled branch, and the same automated pipeline provides objective evidence of the repair.

7.3 Challenges and Resolutions

Challenge	Resolution
Jenkins container lacked Python and Node tooling	Built a custom Jenkins image with Python 3.11, Node 22, npm and Git
College host Python was 3.13 while project CI expected 3.11	Kept the project CI pinned to Python 3.11 inside Docker rather than modifying the application
Existing Jenkins occupied port 8080	Created a separate IntentWay Jenkins container on port 18081
Existing Tomcat occupied port 8081	Mapped the IntentWay Nagios web interface to local port 18080 through a Docker Compose override
First API endpoint test failed in Jenkins	Mocked the final multi-stop route function in the test so external routing remained deterministic
GitHub push initially used the wrong cached account	Re-authenticated the college machine through Git Credential Manager and pushed the correct branch

Table 7.2: Challenges encountered and their resolutions

7.4 Limitations

- The routing result depends on external services and the availability and quality of OpenStreetMap data returned by Nominatim, OSRM and Overpass.
- The greedy solver is a heuristic; it seeks a good route ordering from the current candidate set rather than guaranteeing a global optimum.
- Live map and POI results can vary over time because external geographic data changes.
- The current Jenkins setup demonstrates CI validation; unattended production deployment and rollback are not part of the current repository pipeline.
- Jira and GitHub screenshots capture a specific point in project time. Their counts and branch lists can change as future work is added.
- The monitoring setup is local to the development environment; a production installation would separate monitoring from the application host.

8. Conclusion and Future Scope

8.1 Conclusion

IntentWay demonstrates how a route-planning problem can be treated as a complete software engineering workflow rather than only as a frontend mapping exercise. The application connects natural-language intent parsing, geocoding, road routing, dynamic POI

discovery, spatial filtering, greedy multi-stop optimization, and map-based journey simulation into a single pipeline. The modular design makes each computational stage testable and explainable.

From the Agile and DevOps perspective, the project connects planning and implementation in both directions. Jira provides the work-item view; Git and GitHub provide the source history; Docker provides a repeatable runtime; Jenkins provides automated validation; and Nagios provides service-state monitoring. The evidence captured in the submitted screenshots makes the workflow visible: nine Jira items are complete, multiple Git branches exist, Docker services are running, a Jenkins build is triggered by an SCM change and finishes successfully, and Nagios reports monitored hosts and services.

The central learning from the project is that the individual tools become much more useful when their outputs become inputs to the next stage. A Jira item becomes a branch, a branch becomes a GitHub commit, the commit becomes a Jenkins build, the tested source becomes a Docker runtime, and the running runtime becomes an observable system. IntentWay therefore serves both as a routing project and as a practical demonstration of the Agile and DevOps delivery loop.

8.2 Future Scope

- Introduce a more advanced optimization layer such as beam search, OR-Tools or an exact optimization formulation for small candidate sets.
- Add configurable user preferences such as maximum detour, preferred POI rating, opening hours and accessibility constraints.
- Introduce caching and fallback strategies for geocoding, routing and POI discovery to reduce dependency on external services.
- Add authentication, role-based access, API rate limiting and structured audit logging for a production deployment.
- Extend the simulation with traffic-aware timing and animated stop arrival states.
- Implement WebSockets or server-sent events for live route progress and dynamic route recalculation.
- Extend Jenkins from CI validation to main-branch continuous deployment with smoke tests and rollback.
- Deploy the application to a cloud VM or Kubernetes cluster and run Nagios or another monitoring stack on an independent host.

References

[1] FastAPI Documentation, https://fastapi.tiangolo.com/

[2] Docker Documentation, https://docs.docker.com/

[3] Jenkins Documentation, https://www.jenkins.io/doc/

[4] Git Documentation, https://git-scm.com/doc

[5] GitHub Documentation, https://docs.github.com/

[6] PostGIS Documentation, https://postgis.net/documentation/

[7] Project OSRM, https://project-osrm.org/

[8] Nominatim Documentation, https://nominatim.org/release-docs/latest/

[9] Overpass API Documentation, https://wiki.openstreetmap.org/wiki/Overpass_API

[10] React-Leaflet Documentation, https://react-leaflet.js.org/

[11] Nagios Documentation, https://www.nagios.org/documentation/

[12] Atlassian Jira Documentation, https://support.atlassian.com/jira/

A. Appendix: Running the Project

A.1 Clone and Open the Repository

```
git clone https://github.com/KhanAdnan20-dev/ASD-D-MINIPROJECT.git
cd ASD-D-MINIPROJECT
git checkout main
git pull origin main
```

A.2 Frontend Verification

```
cd frontend
npm install
npm run lint
npm run build
```

The demonstrated college-machine output reported zero frontend lint warnings/errors and a successful Vite production build. The built bundle contained 60 transformed modules, approximately 399.51 kB of JavaScript before gzip, and 36.04 kB of CSS before gzip.

A.3 Docker Compose Verification

```
docker compose config
docker compose up --build
docker compose ps
```

The local college environment used a compose override to expose Nagios on port 18080 because ports 8080 and 8081 were already occupied by existing Jenkins and Tomcat services. This override was kept local so the project repository itself did not need to be changed for the host machine.

A.4 Jenkins Verification

```
docker exec intentway-jenkins python3 --version
docker exec intentway-jenkins node --version
docker exec intentway-jenkins npm --version
docker exec intentway-jenkins git --version
```

The custom Jenkins image used in the demonstration reports Python 3.11.17, Node 22.23.3, npm 10.9.9 and Git 2.47.3. Jenkins was then configured to obtain Jenkinsfile.scripted from the GitHub repository and to poll the selected branch for SCM changes.

A.5 Monitoring Verification

Open the Nagios interface at http://localhost:18080 and use Current Status → Hosts and Current Status → Services to inspect individual checks. The Tactical Overview provides a high-level summary similar to Figure 5.9.

A.6 Evidence Index

Evidence	What it demonstrates
Jira List / Sprint screenshots	Nine completed IntentWay work items, assignees and the completed Sprint 2 scope
GitHub repository screenshot	Repository ownership, main branch, feature branches and recent project history
GitHub collaborators screenshot	Repository access shared with the project collaborators
Git terminal screenshot	Current branch, remote tracking state and recent merge/feature commits
Jenkins console screenshot	SCM-triggered build, branch checkout and pipeline execution
Jenkins success screenshot	94 backend tests passed, frontend build completed, pipeline finished successfully
Docker ps screenshot	Jenkins, frontend, backend, PostGIS and Nagios containers running together
Docker Compose screenshot	Compose network, named Postgres volume and service health state
Nagios Tactical Overview	Four monitored hosts UP, nine services OK and one warning at capture time

Table A.1: Evidence captured for the Agile and DevOps implementation

A.7 Reproducibility Notes

The report evidence is tied to the local environment captured on 06/10/2026. Host-specific items such as the Nagios port override, local Jenkins port, and SCM polling configuration should be treated as environment settings rather than permanent application requirements. The source repository remains the authoritative copy of the application code, Docker Compose definition, Jenkinsfiles and monitoring configuration. Repeating the validation on another machine should begin by cloning the repository, checking the Docker and Node/Python prerequisites, validating docker compose config, and then starting the stack.